

Module 2 – Training Multilayer Perceptrons

Video Transcripts

Video 1: Introduction to the Learning Problem (05:11)

In module one, we introduced neural networks and the problem of training them by minimizing a loss function. We also introduced the method of gradient descent for minimization of functions. In this module, we'll demonstrate how to apply gradient descent to train multilayer perceptrons (MLP).

So, here's our problem of training networks, as we introduced it in module one. We are given a set of training samples. Each training instance includes an input and the target output we desire from the network in response. Our loss function is the average of the divergences between the actual output of the network and the target output. This is the function of the network parameters W . We must find the network parameters, W that minimize this loss. The network is a complicated function of its parameters and so is the loss, so we will use gradient descent to minimize it.

Let's recall the main ideas of the gradient descent algorithm to find the minimum of a function. We start with an initial estimate, and then iteratively take a step against the gradient until we arrive at a minimum. Expanding on this idea, this is our overall gradient descent algorithm to minimize the loss: We initialize our estimate for the optimal value of the network parameters, W . Then we iteratively compute the gradient of the loss function with respect to W at the current estimate of W . We then take a step in the opposite direction of the gradient. That is, we subtract the gradient, multiplied by some learning rate, from the current estimate for the optimal W to obtain the next estimate. We continue the iterations until the loss stops decreasing.

Before we proceed, we have to actually specify the function we are minimizing. Given a training dataset of input-output pairs, we want to minimize the loss. To do that, we have to define every component of the loss. First, what exactly are these input-output pairs? How do we represent them? Second, what is this function $f()$ and what are its parameters? Third, what is the form of this divergence function? Let's start with this question first, since it is at the core of the problem. What is the function $f()$? The function $f()$ is our neural network, a multilayer perceptron.

We will assume that the network is directed, meaning there are no loops. Directed networks use arrows to indicate direction. There's one thing we need to mention here related to terminology: we often refer to the collection of inputs as the input layer, although in reality there are no neurons here, just the inputs. This is convention. The neurons in the last layer, from which we obtain the final outputs, are called output neurons. Neurons at intermediate layers are called the hidden neurons. The term perceptrons, units, and neurons are used interchangeably. Individual neurons each operate on a set of inputs and produce an output which then serves as an input for the next layer.

In the standard setup, a neuron performs two operations. First, the neuron computes an affine function of its inputs. Recall that an affine function is a weighted sum plus a bias. The affine values represented as z in the figure. Then, the neuron applies a continuous activation function $f()$ to this affine value. The parameters of the net are the weights and the biases of all the neurons in it. Here are a number of popular activation functions, and their derivatives.

Historically, the most popular activation function was the sigmoid function. The overall shape goes smoothly from 0 for large negative values of z to 1 for large positive values. Another popular activation function is the tanh, which is just a shifted and scaled version of the sigmoid, so it goes from minus 1 to 1. The derivative of tanh is just a scaled version of the derivative of the sigmoid, though the formula looks very different.

The rectified linear unit, or the ReLU is also very popular. The output equals the input when the input is positive. When the input is negative, the output is 0. The derivative is 0 for negative z and 1 for positive z . At z equals 0, it technically doesn't have a derivative, but you can assume any value between 0 and 1 – this will be a sub derivative. The softplus activation is a smoother version of the ReLU. It avoids the issue of non-differentiability at 0.

Video 2: Problem Setup (03:05)

We'll continue to define the loss function. Let's now complete our definition of the components of this function $f()$. Remember that $f()$ is a network. Using our definition of a layer, we can arrange the network into a series of layers. Only our assumption is that all output neurons are in the same layer. We assumed earlier that each neuron in any layer operates individually. It operates on its inputs and compute a single output; we call such neurons scalar. You can also have neurons that produce multiple outputs. We will call these vector activations – a single neuron receives a set of inputs and outputs a vector of values. Like in the case of individual perceptrons, this situation will have many parameters, but changing any one parameter can now change all outputs.

A very common example of a vector activation is the softmax activation, which operates in two steps. A softmax that computes I outputs, takes in a set of inputs and first produces I different affine terms labeled z here. Then, to compute the actual outputs, the softmax activation exponentiates each of the affine terms and normalizes them to sum to 1. The softmax outputs are all positive and sum to 1 and may be viewed as a probability distribution over I outputs. The parameters of the softmax are the weights and biases of the affine terms. If you change any one weight, such that, say the affine terms z_1 increases, the corresponding y_1 will increase, but because the y values must sum to 1, the other y values will decrease. So, as you can see, changing a single weight changes all the outputs.

Let's take a look at derivatives. We know every affine term z_i affects every output of the softmax. You can compute the derivatives of all the softmax outputs with respect to every affine term. The derivative of the i th output y_i , with respect to its i th affine term z_i is simply

y_i times $(1 - y_i)$. Since y_i is between 0 and 1, this time is positive, showing that increasing z_i will increase y_i . The derivative of any y_j for j not equal to i with respect to z_i is negative $y_i y_j$. This value is always negative and shows that increasing any z_i will decrease the other y_j values. So, now we have defined this function $f()$ and its parameters satisfactorily. Let's move on to the other terms.

Video 3: Derivatives and Gradients (05:12)

Let's continue with defining all terms in our optimization problem. Our next item to address will be these input-output pairs. In defining inputs and outputs, there are three things to consider: the input, the actual output of the network, and the desired output. That is, what we want the network output to be.

For us, an input is a vector, with many components. We represent these inputs by the vector X . The desired output in response to a training input is also a vector, d , with as many components as the number of outputs of the network. The actual network output y will also have as many components as the desired output, d . The network output y is a function of the input and the network parameters.

Each training instance may be shown with two subscripts. The first subscript n represents the index of the training instance. We will generally not explicitly show it. The second subscript represents the component index. For example, $x_{2,5}$ would be the notation for the fifth component of the second input vector. What these numbers represent depends on the type of input. For image input, these numbers will be pixel values. For speech input, these numbers will be a vector of speech features. For text input, these numbers might be a vector of real numbers that represent the text. We'll discuss how to get this from text later in the course.

The representation of the output depends on the problem. If you're trying to compute real-valued outputs, then the output consists of real values. If the desired output is a scalar, like in a scalar regression problem, the network will also have one real-valued output. If the desired output is a vector regression that computes a real-valued vector, then the network will have as many real-valued outputs as the dimensionality of the desired output vector.

If the desired output is binary, like in a binary classification task, we can use a simple 1/0 representation to represent the desired output. The actual output of the network will be computed with a sigmoid activation at the output, which produces an output between 0 and 1. This can be viewed as the a posteriori probability that the output will take value 1. More importantly, it's differentiable. More generally, though, we will simply deal with binary classification as a special case of multi-class classification where there are only two classes.

For networks that perform multi-class classification, to classify inputs into more than one class, we need a slightly more complex representation of the output. If there are N classes, we will construct an N -component vector, where each of the components represents one of the classes. For instance, if we must distinguish an input into one of the following five classes: a

cat, a dog, a camel, a hat, or a flower, we'd list these five classes in some chosen order. Then, an instance of each class becomes a binary vector where all components are 0 except for the component corresponding to the index of that class in the list; that component becomes 1. So, in our arrangement, we have five classes, and the class cat is the first in our order. So, the target output for an instance of cat is a five-component vector where the first component is 1 and the rest are 0. An instance of the class dog is represented by a vector that has a 1 in the second position and 0 elsewhere. Camel is third in our order, so an instance of the class camel is represented by a vector of all 0s except for a 1 in the third position. This is what is called a one-hot vector because only one of the output components is hot, or 1; the rest are cold, or 0.

For a multi-class classifier with N classes, the network output will be an N -component probability vector. That is, it will be an N -component vector of positive numbers that sum to 1. The ideal value of this vector will assign all probability to the true class of the input and 0 to other classes. To obtain the output probability vector in multi-class classification problems, the output layer is generally a softmax vector activation. You get one affine value per class, as we saw earlier. Then we exponentiate all the affine values and normalize them to sum to 1. The output vector therefore represents the vector of probabilities for the classes.

So, we have defined the function $f()$, and how we represent inputs and outputs. We will next define the divergence function.

Video 4: Function Minimization (05:05)

Let's now define the rest of the terms in the loss. We have defined the inputs and outputs and the network, but there's still one more thing to define – the divergence function that quantifies the error between the actual and desired outputs of the network. A very important note: we want to minimize the loss with respect to parameter W using gradient descent. That means the divergence function must be continuous and, ideally, differentiable.

When the desired output of the network is a real-valued vector, like in a regression problem, then the choice for divergence function is the L2 divergence. It is simply half of the squared Euclidean distance between the actual and desired outputs. That is, one half times the sum overall output components of the square of the difference between the network output Y and the desired output d . This factor of one half is only a matter of convenience to cancel out the factor of 2 we get when we take the derivative of the square.

The divergence is a scalar number. The derivative of the divergence with respect to any single output component is simply the error y minus d . So, assuming the output of the network to be a column vector, the derivative of the divergence with respect to the output vector will be a row vector whose components are the error. That is, the components are the differences between the corresponding components of y and d . For classification, the desired output d will be a one-hot vector. The network output y is a vector of probabilities. We won't distinguish

between binary and multi-class classification here, because we consider binary classification to be a special two-class case of multi-class classification.

The KL divergence between the network output Y and the desired output d is given by this formula: the sum over all classes of $d_i \log d_i$ minus $d_i \log y_i$. When d is a one-hot vector, $d_i \log d_i$ is always 0 because either d_i is 0, or d_i is 1, in which case $\log d_i$ is 0. So, the first term, $\sum d_i \log d_i$, becomes 0. Also, the d_i 's for the non-target class are 0, and for the target class alone, d_i is 1. So, this entire term reduces to $-\log y_c$, where y_c is the probability assigned to the target class. So, the KL divergence simply becomes the negative of the log probability assigned to the target class.

The derivative of the divergence with respect to the individual outputs will be 0 for the non-target classes because they don't figure in the equation. For the target class, the derivative of the divergence with respect to y_c will be equal to $-1/y_c$. So, the vector derivative of the divergence with respect to Y will be a row vector, whose components are all 0, except for the component corresponding to the target class, which is $-1/y_c$. Observe that the derivative is negative, the divergence decreases if y_c increases. Minimizing this divergence will try to increase the probability of the correct class, which is as it should be.

The second term in the KL divergence is the cross-entropy between d and Y . If d is one-hot, the first term is 0, so the KL divergence equals the cross-entropy, so we often refer to this as the cross-entropy loss. Even if d_i is not one-hot, the first term in the KL divergence is not a function of Y , so minimizing the cross-entropy with respect to Y will also minimize the KL divergence. So usually, we will minimize the cross-entropy rather than the KL divergence. Note that, if d is not one-hot, cross-entropy by itself is not a divergence and, although it will be minimum when Y equals d , it won't be 0.

An interesting note – regression models generally do not use an activation at the output, so the output y is equal to the affine value z . Classification models apply a softmax to z to compute y . We usually compute the L2 divergence for regression models and the KL divergence or cross-entropy loss for classifiers. In both cases, the gradient of the loss with respect to the affine value z will simply be the error, y minus d . It is often just simpler to work with the gradient of the loss with respect to z directly. So now, we have a complete problem specification – all required terms have been defined.

Video 5: Training a Neural Network (04:53)

Having defined the problem, now let's actually work out gradient descent to train the network. Here again is our problem setup. We are given a training set of input-output pairs. We define our loss as the average divergence between the actual and desired outputs of the net for the training set. I am representing the output of the net in response to the i th input as y_i . We want to minimize the loss with respect to the weights and biases of all the neurons in the network, and we'll use gradient descent for it.

Let's take a look at some notation. The input layer is the 0th layer. This is just the input to the network. We will represent the output of the i th neuron of the l th layer as y_i^l . The superscript represents layer number, and the subscript is the neuron index in the layer. So, y_i^0 which is the i th value at the 0th layer is simply the i th input component.

The output of the network y_i is simply the i th output component of the final N th layer Y_i^N . We will represent the weight of the connection between the i th unit of the $(l-1)$ th layer and the j th unit of the l th layer as W_{ij}^l . The subscripts indicate the source and destination neurons of the connection, and the superscript is the destination layer. So, $W_{1,2}^3$ is the weight of the connection between the first neuron of the second layer to the second neuron of the third layer.

Using the same notation, the bias to the j th neuron of the l th layer, is b_j^l . So, $b_{2,3}$ is the bias of the second neuron in the third layer. So, here again is the gradient descent algorithm. This is the loss we will minimize – the average divergence over the training data. I'm not showing the dependence on W for brevity, but loss is a function of the parameters.

We will first initialize all the parameters of all the layers in the network. Then, for every layer in the network, we will update every single weight to every single neuron, by shifting it in the direction opposite to the derivative of the loss with respect to that weight. In this notation here, I'm assuming that the bias is also represented as a weight, which we saw we could do if we simply appended an extra input to each neuron with the value fixed to 1.

To implement this, we would need to be able to compute the derivative of the loss with respect to the individual network parameters. The loss, of course, is the average of the divergences for the individual training instances. So, the derivative of the loss with respect to the individual weights is the average of the derivatives of the divergences for the individual training instances with respect to the weights.

The key problem is this – how to compute the derivative of the divergence for a single data instance, with respect to all network parameters. I'm going to use this tiny 2-input network to illustrate the details of the computation. An actual network would, of course, have many more neurons and inputs. To make it more explicit, I have also shown all the weights. I'm also adding an extra unit 1 in each layer. The weight of the connection from this unit is known as the bias, which was introduced previously. In each neuron, there are actually two computation steps. In the first step, the neuron computes affine function of inputs, which is represented by the symbol z . In the second step, the neuron applies an activation function to the affine values z .

Let's also label the remaining variables, f_i is the activation function for the i th layer neurons. z_j^l is the affine combination computed at the j th neuron of the l th layer. Y_j^l is the output of the j th neuron of the l th layer. We are showing the variables for a single input, X is the input, d is the desired output for X , y is the actual output of the net, and the box marked Div represents the divergence. We want to compute the derivative

of this divergence with respect to any weight in the net, which could be as far back as the first layer.

Video 6: Calculus Refresher: Chain Rule (04:59)

Before we continue, let's go through a calculus refresher. The derivative of a differential function, y equals f of x , at any point is the multiplicative factor by which you must multiply a tiny increment in x to obtain the corresponding increment in y . So, for any sufficiently small increment, Δx , the corresponding increment Δy equals the derivative times Δx . We can show this pictorially using what I would call, an influence diagram. x influences y as shown. So, by the rules of calculus, we get the relationship described in the diagram. The input to the edge is Δx . The weight of the edge is the derivative, dy over dx . So, accordingly, the increment Δx is weighted by the derivative dy over dx , to give you the increment Δy . So, Δy equals dy over dx times Δx .

So, also for a multivariate function of many variables, x_1 through x_M , y equals f of x_1 through x_M , we have the following rule relating the increments Δy to the corresponding increments in the input: Δy is the sum of the partial of y with respect to x_1 times Δx_1 plus the partial of y with respect to x_2 times Δx_2 and so on, summed over all components. This too can be shown through an influence diagram. As shown, x_1 through x_M , all influence y . This gives us the derivative figure. Each edge now carries a weight that is the partial derivative of y with respect to the input variable. Therefore, Δy is the sum of the contributions over all incoming edges, which gives us Δy equals Δx_1 times the partial of y with respect to x_1 , plus Δx_2 times the partial of y with respect to Δx_2 , and so on until x_M times the partial of y with respect to x_M . Now, consider a nested function, y equals f of $g(x)$.

The chain rule we learned in school says, dy over dx equals dy over dg times dg over dx . This too is obtained through the influence diagram. As shown, x influences g because g is a function of x . g in turn influences y . And the derivative figure, the edges carry the derivative of the output variable of the edge with respect to the input variable. So, we get Δy equals Δg times dy over dg . Δg in turn equals Δx times dg over dx . So, the derivative of y with respect to x is the product of the weights of all edges on the path from x to y , which is the product of the two derivatives, dy over dg times dg over dx . Let's consider a more complex case where y is a function of many different functions of x , so y equals f of g_1 of x , g_2 of x , and so on until g_M of x . If I represent g_i of x as z_i , we have x influencing each of z_1 through z_M , each of which in turn influences y .

So, from the figure, Δy has additive contributions from each of the incoming paths, and so increments in y have contributions from all incoming paths – Δy equals the sum over all i of the partial of y with respect to z_i times Δz_i . Each z_i , in turn, is only dependent on x , so we get Δz_i equals dz_i over dx times Δx . Combining all of these, we see the derivative of y with respect to x : dy over dx equals the partial of y with respect to z_1 times dz_1 over dx

plus the partial of y with respect to z_2 times dz_2 over dx and so on until we add partial of y with respect to z_M times dz_M over dx . Remember these relations.

Video 7: Vector Calculus 1 (03:26)

Before we see how the backward pass is done using vectors, let us quickly review vector calculus. Remember this definition: the derivative of a function at any input is the multiplication factor that multiplies and increment in the input to compute the corresponding change in the output. This also holds for vectors. So, if we have a scalar function y equals f of z , where y is a scalar and z is a vector, then the increment Δy is the derivative of y with respect to z times Δz . Δy is a scalar, so if z is a column vector, the derivative is a row vector of the same size.

And, of course, the transpose of the derivative is the gradient, as we saw earlier. If y is also a vector, then the relationship still holds. Δy is the derivative times Δz . So, if z is an R times 1 vector, and y is an L times 1 vector, then clearly the derivative shape must be an L times R matrix.

The derivative of a vector function of a vector argument is a matrix called its Jacobian. A Jacobian is the derivative of a vector function of a vector variable, with respect to its input vector. This is represented as $J_{\text{subscript } y \text{ of } z}$. The notation says we are computing the derivative of y with respect to z at a specific location z . The Jacobian is a matrix. Each row of the Jacobian is the row of partial derivatives of the corresponding component of y with respect to the components of z . Basically, the i th row of the Jacobian is the derivative of the i th component of y with respect to z . And, of course, this relationship will hold: for any infinitesimal increment Δz , the corresponding increment Δy is given by the Jacobian times Δz .

Jacobians can describe the derivatives of neural activations with respect to their input. In vector notation, the activation for each layer of a network takes in a vector z and outputs a vector y . So, the derivative of y with respect to z will be a Jacobian. If the activations of the layer are scalar activations where the output of each neuron in a layer only depends on its local affine input, then the Jacobian will be a diagonal matrix, where the diagonal elements are simply f' of z_i .

If the activation function for the layer is a vector activation, like a softmax, then the Jacobian will be a full matrix, with as many rows as outputs and as many columns as affine inputs. The i th row will be a row of partial derivatives for the i th output with respect to each of the affine inputs. One vector activation we will often encounter is the softmax. Its Jacobian is a square matrix. The i th diagonal entry is y_i times $1 - y_i$. The off diagonal ij th entries are all $-y_i y_j$.

Video 8: Vector Calculus 2 (05:10)

For the affine combinations that occur in the forward pass, we have this relation: z equals W_y plus b , where y is the output vector of one layer, and z is the affine value vector of the next layer. W and b are the weights and biases. So, z is an affine function of y . In this case, you can see that if you increment y by Δy , the corresponding change Δz will simply be W times Δy . So, the Jacobian of z with respect to y is simply this matrix W .

Now consider a nested function of this kind: All terms here x , y , z maybe a scalar or a vector. Again, we are assuming column vector convention. The chain rule states that the derivative of y with respect to x is the derivative of y with respect to z times the derivative of z with respect to x . Note the order over here: the first term is the derivative of y with respect to its argument, the second term is the derivative of the inner term z of x with respect to its argument, so the derivatives chain outside in. As you go into the nested function, the derivatives for the inner functions keep getting appended to the right here. We can verify this easily.

The inverted triangle symbol, nabla, is used to represent derivatives. So, $\nabla_z y$ is the derivative of y with respect to z . By the definition of derivatives, we know that, because y is a function of z , Δy equals $\nabla_z y$ times Δz for small increments in z . $\nabla_z y$ is the derivative of y with respect to z . We also know that z is a function of x , so Δz is $\nabla_x z$ times Δx . Chaining the two, we get Δy equals $\nabla_z y$ times $\nabla_x z$ times Δx . So, this tells us that the derivative of y with respect to x is the derivative of y with respect to z times the derivative of z with respect to x .

We will encounter the chain rule twice. The first is at activations. Consider the activation at any layer: we have y equals f of z , where z is the affine value going in and y is the computed output. The final divergence is a scalar that is derived from y . By the chain rule, the derivative of the divergence with respect to z is the derivative of the divergence with respect to y times the derivative of y with respect to z . Again, note the order of computation: the derivative of y with respect to z is just the Jacobian of the activation, so the derivative of the divergence with respect to z is the derivative of the divergence with respect to y times the Jacobian of the activation.

The next use of the chain rule is at affine functions. At the k th layer the affine value z_k is an affine function of the output of the previous layer y_{k-1} . So, z_k equals W_k times y_{k-1} plus b_k , where W_k and b_k are the weights and bias for the k th layer. So, by earlier rule, the derivative z_k with respect to y_{k-1} is just W_k . The divergence is eventually a function of z_k . So, by the chain rule, the derivative of the divergence with respect to y_{k-1} is this product: the derivative of the divergence with respect to z_k times the derivative of z_k with respect to y_{k-1} which is just the derivative of the divergence with respect to z_k times the weight matrix W_k .

Also, within the affine layer, the derivative of the bias with respect to z is just an identity matrix. So, the derivative of the divergence D with respect to bias b_k it's just the derivative of the divergence with respect to z_k itself. This last equation is more complex. This is for the

derivative of the divergence with respect to the weight matrix W_k . I won't show how this is derived, but it's just y_k minus 1 times the derivative of the divergence with respect to z_k . Note the order over here: it is the product of the input to the affine function times the derivative of the divergence with respect to the output of the affine function. It'll have the same shape as the transpose of W . And now we have everything we required to perform our backward computation.

Video 9: Training by Backpropagation (03:49)

Having worked through how to perform forward computation for inference and the backward pass to compute derivatives, let's put it all together. Here is the pseudocode for the backward pass. We first compute the derivative of the divergence with respect to the network output. We then, iterate backwards to the first layer. At each layer, we first compute the Jacobian for the activation.

Then, we write multiply the derivative for y_k by the Jacobian to get the derivative for Z_k . We multiply that result by the weight matrix to get the derivative with respect to Y_k minus 1. We then get the derivative for the weight as the product of Y_k minus 1 and ∇z_k , and the derivative for the bias is ∇z_k . We compute these backwards through the network until the first layer. Note that as we go backwards, we always have all the variables required to compute the next derivative. This process of going backward through the net to compute derivatives is called backpropagation; this is short for error backpropagation or just backprop. This process is used to compute the derivatives of the divergences for individual training instances, and it is used in gradient descent.

So, here is the overall procedure for training via backdrop. We first initialize the weights and biases for all layers. Then, we iterate. At each iteration, we go over all training instances. For each instance, we first perform a forward pass, compute the output and the divergence, and update the loss for the iteration. Then, we perform a backward pass where we compute all the intermediate derivatives and the derivatives with respect to all network parameters. Once we've gone over all training instances, we go through the net and update the weights and biases for each layer using the gradient decent rule, using the average derivative over all training instances. We continue this until the average loss over the training data converges.

Let's look at an example of the process for setting up for a typical multiclass classification problem such as digit recognition. We are given images of digits and we must recognize the digit. Our network will have 10 outputs, one per digit. We may have a 11th output to indicate if the input is not a digit at all. The output layer would be a softmax layer. The training data will be the images of digits, with their desired labels, which we will represent as one-hot vectors. So, our desired output will be one-hot vectors. We will use the KL divergence to quantify the network error, and backpropagation to compute derivatives to iteratively update the network parameters to learn them.

The story so far then, is that neural networks must be trained to minimize the average divergence between the output of the network and the desired output over a set of training instances, with respect to network parameters. Minimization is performed using gradient descent. Gradients which are the transposes of derivatives of the divergence for any individual instance, with respect to network parameters, can be computed using backpropagation. Gradient computation requires a forward pass of inference, and by inference, I mean computing the network output, followed by a backward pass of gradient computation. The computed gradients can be used for gradient descent-based updates of parameters.